

Two things Genesis does that cost AMD users time

Both were found while building Chaal on a Radeon Cloud instance (gfx1100, ROCm 7.2.4, Genesis 1.3.0). Both are reproducible on any machine. The first stops a headless ROCm container from running at all; the second does something worse, which is to run fine and give you a wrong number.

1. A scene cannot be built without an OpenGL context, even with nothing to render

A ROCm container is normally built without any OpenGL libraries, because nothing in the compute stack needs them. Genesis builds an offscreen renderer anyway, during `scene.build()`, whether or not anything is being rendered.

Minimal reproduction on a container with no `libEGL`:

```
import genesis as gs
gs.init(backend=gs.amdgpu)
scene = gs.Scene(show_viewer=False)          # no viewer
scene.add_entity(gs.morphs.Plane())          # no cameras
scene.build(n_envs=1)
```

```
File "genesis/vis/rasterizer.py", line 40, in build
    self._renderer = pyrender.OffscreenRenderer(
File "OpenGL/raw/EGL/_types.py", line 87, in <module>
    raw_eglQueryString = _p.PLATFORM.EGL.eglQueryString
AttributeError: 'NoneType' object has no attribute 'eglQueryString'
```

Nothing in that message says "you have no OpenGL platform, install libEGL".

Why there is no way around it. `Visualizer.__init__` sets

```
self._context = RasterizerContext(vis_options)
```

unconditionally, and `Visualizer.build()` then calls `self._rasterizer.build()` unconditionally. `Rasterizer.build()` returns early only if `self._context` is `None`, which after construction it never is. `show_viewer=False` skips the interactive viewer only. Neither `Scene.__init__` nor `VisOptions` exposes any flag to disable rendering.

This matters most on exactly the machines Genesis is good on: a headless GPU box running state-based RL, where rendering is not wanted and the GL libraries are not installed.

Workaround. Install a GL platform even though nothing will be drawn:

```
apt-get install -y libegl1 libegl-mesa0 libgl1-mesa-dri libosmesa6
```

Suggested fix. Either let `Rasterizer.build()` be skipped when no camera has been added and no viewer is shown, or catch the failure and re-raise it with the cause and the apt line above.

2. `set_friction()` on one entity silently does nothing

This is the expensive one, because it does not raise, it does not warn, and the run completes with plausible numbers.

To test a policy on a slippery floor the obvious code is:

```
robot.set_friction(0.2)
```

That does change the robot's own geoms, from 1.0 to 0.2, and it changes contact friction not at all, because **Genesis resolves a contact pair by taking the larger of the two geoms' friction values** and the ground plane is still at 1.0.

Measured, same policy and same seed, 512 environments, 10 simulated seconds:

robot mu	plane mu	survival	tracking error
1.0	1.0	1.000	0.0972 m/s
0.2	1.0	1.000	0.0973 m/s
1.0	0.2	1.000	0.0960 m/s
0.2	0.2	0.898	0.2977 m/s

Setting either surface alone reproduces the baseline to three decimal places. Setting both drops survival by ten points and triples tracking error.

The failure mode is publishing "our policy is robust to low friction" on the strength of the second row, which is the baseline measured twice. Any domain-randomisation sweep over friction that randomises only the robot has this bug, and nothing in the output shows it.

Suggested fix. Document the max-combination rule on `set_friction`, and consider warning when an entity's friction is set well below that of another entity it is in contact with.

Correct usage, for anyone who lands here first

```
robot.set_friction(mu)
plane.set_friction(mu) # both, or it does nothing
```